# CHAPTER 14

# DESIGN



# 14.1 Design and Abstraction

- Read through this section!



# 14.2 Operation-Oriented Design

- Read through this section!



# 14.3 Data Flow Analysis

- Read through this section!

# 14.4 Transaction Analysis

- Read through this section!

# 14.5 Data-Oriented Design

- Read through this section!



# 14.6 Object-Oriented Design (OOD)

- OOD (Object-Oriented Design) consists of two steps:
- Step 1. Complete the class diagram
  - Determine the formats of the attributes
  - Assign each method, either to a class or to a client that sends a message to an object of that class
- Step 2. Perform the detailed design

# Object-Oriented Design Steps (contd)

- Step 1. Complete the class diagram
  - The formats of the attributes can be directly deduced from the analysis artifacts
- Example: Dates
  - U.S. format (mm/dd/yyyy)
  - European format (dd/mm/yyyy)
  - In both instances, 10 characters are needed

# Object-Oriented Design Steps (contd)

- The formats could be added during analysis
  - Object-oriented paradigm is iterative
  - Each iteration results in a change to what has already been completed
  - To minimize rework, *never* add an item to a UML diagram until strictly necessary



# Object-Oriented Design Steps (contd)

- Step 1. Complete the class diagram
  - Assign each method, either to a class or to a client that sends a message to an object of that class
  - Do this by examining the interaction diagrams

# Estimate Funds Available for Week Use Case (contd)

- Communication diagram (of the realization of the scenario of the use case)

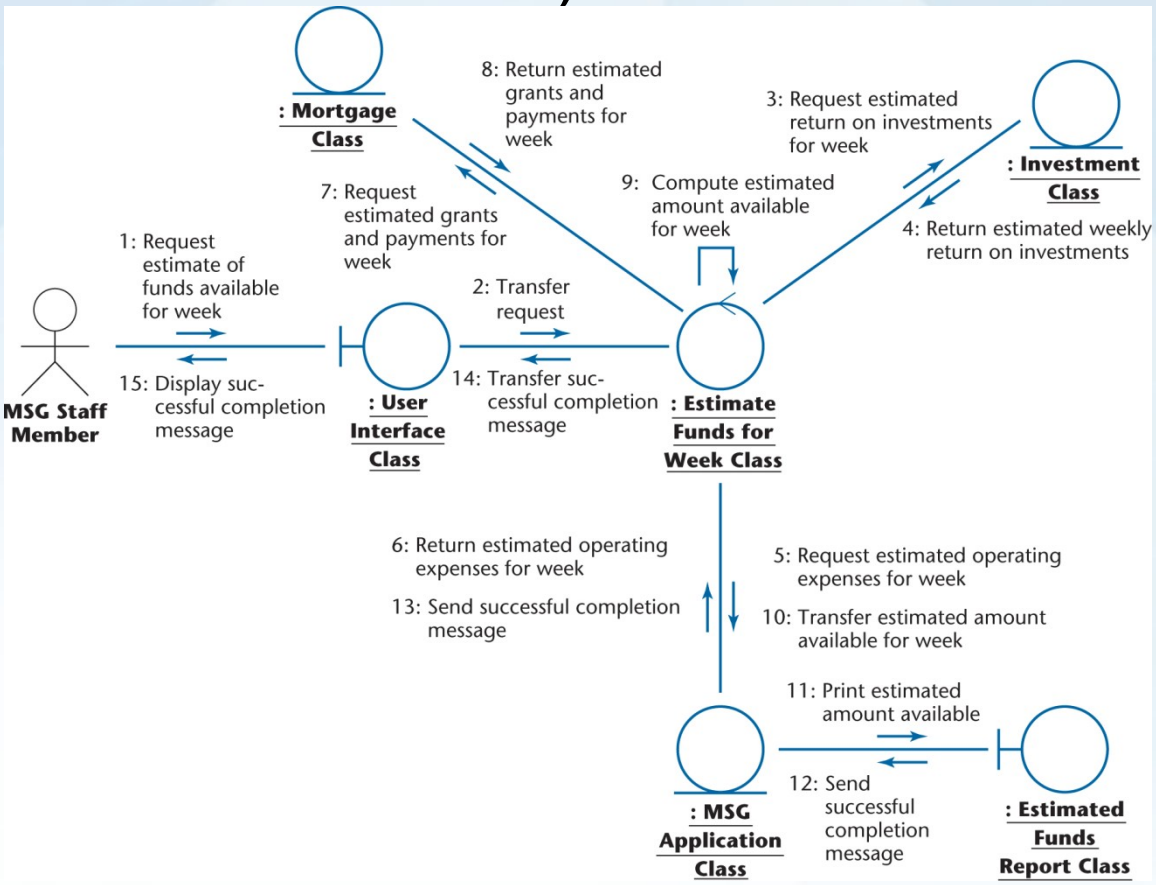


Figure 13.36

# Object-Oriented Design Steps (contd)

- The difficulty is on deciding which method to assign to which class
- **Principle A:** Information hiding
  - All variables are declared private or protected and therefore operations performed on those variables must be local to the class



# Object-Oriented Design Steps (contd)

- **Principle B:** If an operation is invoked by many clients of an object, assign a single copy to the object, rather than have a copy in each client
- **Principle C:** Responsibility-driven design
  - Client sends a message to an object,
  - That object is responsible for every aspect of carrying out the request of the client
  - Client does not know how the request will be carried out
  - Once request has been carried out, control is returned to client

# 14.7 Object-Oriented Design: The Elevator Problem Case Study

- Step 1. Complete the class diagram
  - Consider the second iteration of the CRC card for the elevator controller

| CLASS<br>**Elevator Controller Class** |
|---|
| RESPONSIBILITY<br>1. Send message to **Elevator Button Class** to turn on button<br>2. Send message to **Elevator Button Class** to turn off button<br>3. Send message to **Floor Button Class** to turn on button<br>4. Send message to **Floor Button Class** to turn off button<br>5. Send message to **Elevator Class** to move up one floor<br>6. Send message to **Elevator Class** to move down one floor<br>7. Send message to **Elevator Doors Class** to open<br>8. Start timer<br>9. Send message to **Elevator Doors Class** to close after timeout<br>10. Check requests<br>11. Update requests |
| COLLABORATION<br>1. **Elevator Button Class** (subclass)<br>2. **Floor Button Class** (subclass)<br>3. **Elevator Doors Class**<br>4. **Elevator Class** |

Figure 13.9 (again)

# Detailed Design: Elevator Problem

- Detailed design of `elevatorEventLoop` is constructed from the statechart

```
void elevatorSubcontrollerEventLoop (void)
{
  while (TRUE)
  {
    if (an elevatorButton has been pressed)
        if (elevatorButton is off)
        {
                elevatorButton::turnOnButton;
                scheduler::newRequestMade;
        }
    else if (elevator is moving up)
    {
        wait for sensor message that elevator is arriving at floor;
        scheduler::checkRequests;
        if (there is no request to stop at floor f)
            elevator::moveUpOneFloor;
        else
        {
            stop elevator by not sending a message to move;
            if (elevatorButton is on)
                elevatorButton::turnOffButton;
            elevatorDoors::openDoors;
            startTimer;
    }
  }
  else if (elevator is moving down)
        [similar to up case]
  else if (elevator is stopped and request is pending)
  {
        wait for timeout;
        elevatorDoors::closeDoors;
        determine direction of next request;
        elevator::moveUp/DownOneFloor;
        wait for sensor message that elevator has left floor;
        floorSubcontroller::elevatorHasLeftFloor;
    }
    else if (elevator is at rest and not (request is pending))
    {
        wait for timeout;
        elevatorDoors::closeDoors;
    }
    else
        there are no requests, elevator is stopped with elevatorDoors closed, so do nothing;
  }
}
```

Figure 14.12

# 14.9 The Design Workflow

- **Summary of the design workflow:**
  - The analysis workflow artifacts are iterated and integrated until the programmers can utilize them
  - Identification of the methods and the allocation to classes
  - Detailed design is also performed
- **Decisions to be made include:**
  - Implementation language
  - How much of the existing software must be reused
  - Portability

# The Design Workflow (contd)

- The Unified Process was designed to present a methodology that could be used to develop large-scale software products
- Analysis workflow is used to partition the system into packages
  - A package is a set of related classes, usually of relevance to a subset of actors, that can be implemented as a single unit



# The Design Workflow (contd)

- The idea of decomposing a large workflow into independent smaller workflows (*packages*) is carried forward to the design workflow
- The objective is to break up the upcoming implementation workflow into manageable pieces
  - *Subsystems*

# The Design Workflow (contd)

- Why the product is broken into subsystems:
  - It is easier to implement a number of smaller subsystems than one large system
  - If the subsystems are independent, they can be implemented by programming teams working in parallel
    - The software product as a whole can then be delivered sooner



# The Design Workflow (contd)

- The *architecture* of a software product includes
  - The various components
  - How they fit together
  - The allocation of components to subsystems
- The task of designing the architecture is specialized
  - It is performed by a software *architect*



# The Design Workflow (contd)

- The architect needs to make *trade-offs*
  - Every software product must satisfy its functional requirements (the use cases)
  - It also must satisfy its nonfunctional requirements, including
    - Portability, reliability, robustness, maintainability,
  - It must do all these things within budget and time constraints
- The architect must assist the client by laying out the trade-offs

# The Design Workflow (contd)

- It is usually impossible to satisfy all the requirements, functional and nonfunctional, within the cost and time constraints
  - Some sort of compromises have to be made
- The client has to
  - Relax some of the requirements;
  - Increase the budget; and/or
  - Move the delivery deadline

# The Design Workflow (contd)

- The architecture of a software product is critical
  - The requirements workflow can be fixed during the analysis workflow
  - The analysis workflow can be fixed during the design workflow
  - The design workflow can be fixed during the implementation workflow
- But there is no way to recover from a suboptimal architecture
  - The architecture must immediately be redesigned

# 14.10 The Test Workflow: Design

- Design reviews (inspections or walkthroughs) must be performed
  - The design must correctly reflect the specifications
  - The design itself must be correct
- Transaction-driven inspections
  - Essential for transaction-oriented products
  - However, they are insufficient — specification-driven inspections are also needed to detect whether the designers have overlooked instances of transactions required by the specifications

# 14.12 Formal Techniques for Detailed Design

- Implementing a complete product and then proving it correct is hard
- However, use of formal techniques during detailed design can help
  - Correctness proving can be applied to module-sized pieces
  - The design should have fewer faults if it is developed in parallel with a correctness proof
  - If the same programmer does the detailed design and implementation
    - The programmer will have a positive attitude to the detailed design
    - This should lead to fewer faults



# 14.13 Real-Time Design Techniques

- Read through this section!



# 14.15 Metrics for Design

- Read through this section!



# 14.16 Challenges of the Design Phase

- Designers must do the right amount of work
  - Designers may write the detailed code instead of just the PDL
  - Designers may provide too little information in the detailed design, and leave the detailed design to the programmers



# Challenges of the Design Phase (contd)

- We need to “grow” great designers

- Potential great designers must be
  - Identified,
  - Provided with a formal education,
  - Apprenticed to great designers, and
  - Allowed to interact with other designers

- There must be a specific career path for these designers, with appropriate rewards